German University in Cairo
Faculty of Engineering & Materials Science – Mechatronics Department
MCTR911 – Robotics Programming – Winter 2026

Milestone 02 Report

UR5e Kinematics (DH, Forward & Inverse Position), ROS 2 – MuJoCo Integration, Motion Inputs and GUI

1. Objective and Milestone 02 checklist

This report documents the second milestone of the MCTR911 project. The goal of this milestone is to model the chosen manipulator kinematically, implement forward and inverse position kinematics in Python, place the robot model in MuJoCo and link it to ROS 2, drive its joints with constants, signal builders and sliders, and begin building the GUI of the environment. The table below maps every requirement of the project description to the section of this report and the files that satisfy it.

Requirement	Where	Evidence
Assign frames on the arm	Section 3	MuJoCo frame visualisation (Figures 1–2)
Develop the DH convention	Section 4	DH table and transform matrix
Forward position kinematics (Python)	Section 5	`kinematics_ur5e.py`
Inverse position kinematics (Python)	Section 6	`kinematics_ur5e.py` (damped least squares)
Place model in MuJoCo, link with ROS 2	Section 7	`mujoco_actuation_ur5e.py`
Actuate with constants, signal builders, sliders	Section 8	Bridge modes, GUI sliders, MuJoCo viewer sliders
Analyse and visualise the motion	Section 8	Signal analysis (Figure 7)
Start building the GUI	Section 9	`New_ur5e_gui.py`

2. The robot: UR5e

The UR5e is a six-axis collaborative industrial arm with a 5 kg payload and about 850 mm reach. All six joints are revolute, so the joint vector is $q = [\theta_1 \dots \theta_6]$. The first three joints (base, shoulder, elbow) position the wrist; the last three joints (wrist 1, 2, 3) orient the tool flange. The simulation model is the UR5e from the MuJoCo Menagerie collection (`mujoco_menagerie/universal_robots_ur5e/scene.xml`), which provides meshes, joint limits and position actuators for all six joints.

Joint	Name in ROS / MuJoCo	Role
1	shoulder_pan_joint	Rotates the whole arm about the vertical base axis
2	shoulder_lift_joint	Raises or lowers the upper arm

Joint	Name in ROS / MuJoCo	Role
3	elbow_joint	Bends the forearm relative to the upper arm
4	wrist_1_joint	First wrist rotation (pitch)
5	wrist_2_joint	Second wrist rotation
6	wrist_3_joint	Tool flange rotation (roll about the tool axis)

3. Frame assignment

Frames were assigned following the standard (distal) Denavit–Hartenberg rules: the Z axis of frame {i−1} lies along the axis of joint i; the X axis of frame {i} lies along the common normal between Z_{i-1} and Z_i (pointing away from the previous axis); and Y completes the right-handed set. Frame {0} is the base frame and frame {6} is attached to the tool flange. The frames were visualised directly in the MuJoCo viewer (Rendering → Frame → Body), where red, green and blue arrows are the X, Y and Z axes.

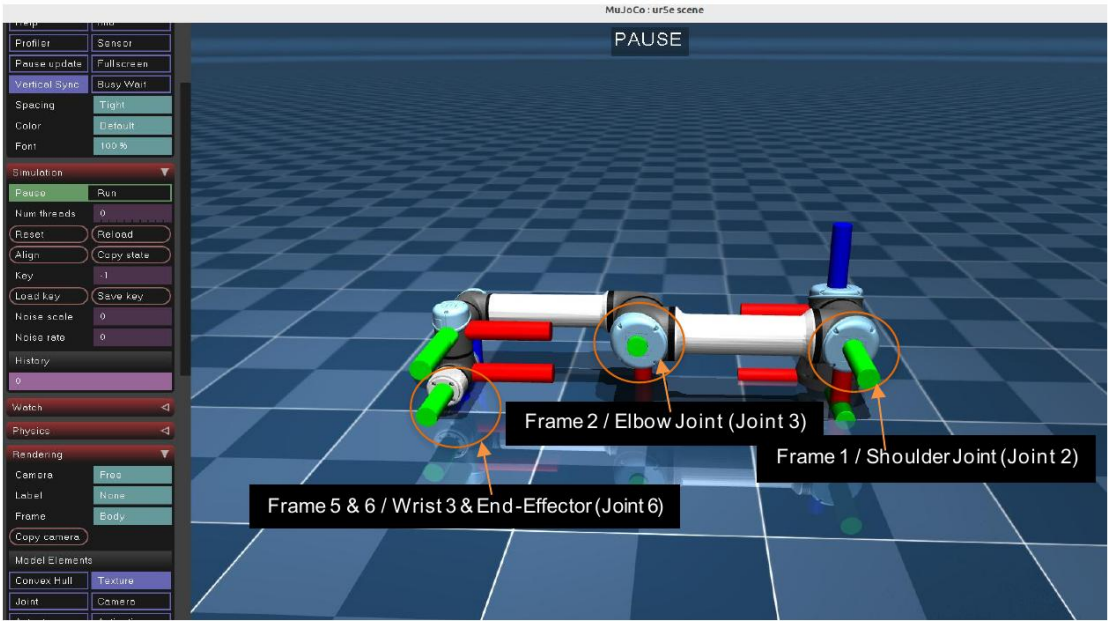


Figure 1. Side view of the UR5e in MuJoCo with the body frames displayed and labelled (red = X, green = Y, blue = Z).

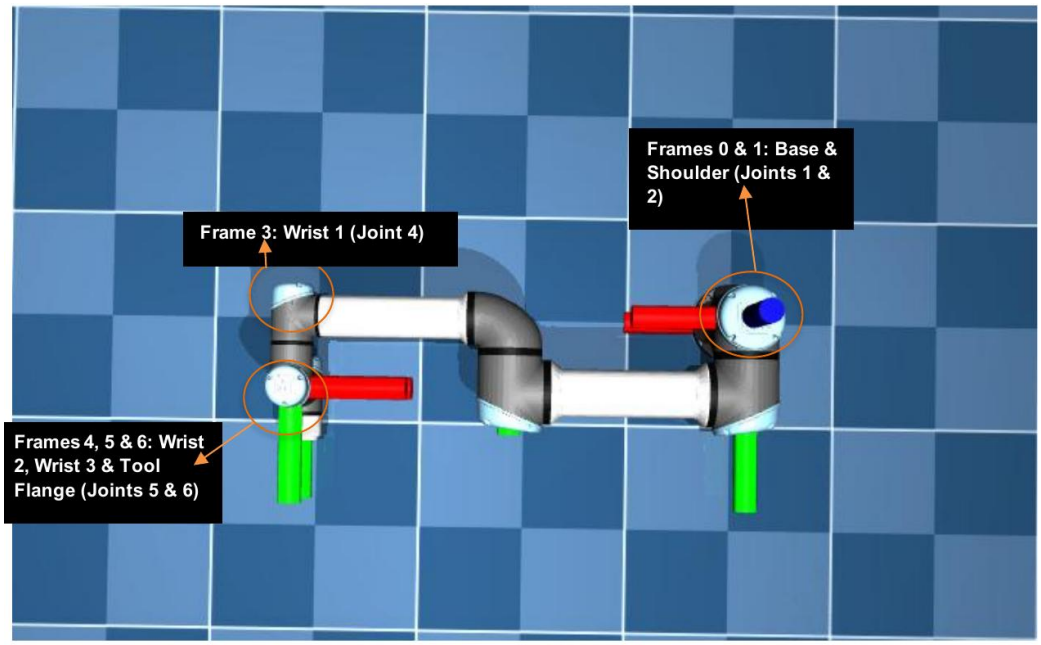


Figure 2. Top view of the same pose with the frames labelled. The base and shoulder frames are at the right, the wrist 1 frame at the upper left, and the wrist 2, wrist 3 and tool-flange frames at the lower left.

Observations from the frames: the shoulder and elbow joint axes are parallel (both horizontal), which is why the link twists α_2 and α_3 are zero and why the upper arm and forearm lie in a single plane. The three wrist axes intersect pairwise at right angles ($\alpha_4 = +\pi/2$, $\alpha_5 = -\pi/2$), giving the usual spherical-like wrist arrangement with small offsets d_4, d_5 and d_6.

4. Denavit–Hartenberg convention

4.1 Parameter definitions

Each transformation from frame {i−1} to frame {i} is described by four standard DH parameters:

- **θ_i (joint angle):** rotation about Z_{i-1} that aligns X_{i-1} with X_i. This is the variable of each revolute joint.
- **d_i (link offset):** translation along Z_{i-1} from O_{i-1} to the point where the X_i axis intersects Z_{i-1}.
- **a_i (link length):** translation along X_i, equal to the length of the common normal between Z_{i-1} and Z_i.
- **α_i (link twist):** rotation about X_i that aligns Z_{i-1} with Z_i.

4.2 DH parameter table

i	Joint	θ_i (rad)	d_i (m)	a_i (m)	α_i (rad)
1	Base	θ_1*	0.1625	0	π/2
2	Shoulder	θ_2*	0	−0.4250	0
3	Elbow	θ_3*	0	−0.3922	0
4	Wrist 1	θ_4*	0.1333	0	π/2
5	Wrist 2	θ_5*	0.0997	0	−π/2
6	Wrist 3	θ_6*	0.0996	0	0

Table 1. Standard DH parameters of the UR5e (= joint variable). Values follow the manufacturer's published kinematic parameters.*

4.3 Transformation matrices

The elementary operations are composed in the order rotate about Z, translate along Z, translate along X, rotate about X:

$$T(i{-}1{\to}i) = \mathrm{Rot}(Z, \theta_i) \cdot \mathrm{Trans}(Z, d_i) \cdot \mathrm{Trans}(X, a_i) \cdot \mathrm{Rot}(X, \alpha_i)$$

$$\begin{vmatrix} c\theta & -s\theta{\cdot}c\alpha & s\theta{\cdot}s\alpha & a{\cdot}c\theta \\ s\theta & c\theta{\cdot}c\alpha & -c\theta{\cdot}s\alpha & a{\cdot}s\theta \\ 0 & s\alpha & c\alpha & d \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

The pose of the tool flange relative to the base is the product of the six link transforms:

$$T_{06} = T_{01}(\theta_1) \cdot T_{12}(\theta_2) \cdot T_{23}(\theta_3) \cdot T_{34}(\theta_4) \cdot T_{45}(\theta_5) \cdot T_{56}(\theta_6)$$

The upper-left 3×3 block of T_{06} is the tool orientation and the right-hand column gives the tool position (x, y, z). The implementation is the function `dh_transform(theta, d, a, alpha)` in `kinematics_ur5e.py`; the DH table is stored in the list `DH_PARAMS` as [d, a, α] per joint.

5. Forward position kinematics

`forward_kinematics(q)` multiplies the six DH matrices in order and returns T_{06} together with the origin of every frame (used for plotting). A second helper, `forward_kinematics_ur5e(q)`, returns the cumulative transform of each frame, and `get_end_effector_position(q)` returns only (x, y, z).

5.1 Analytical check at the zero configuration

At q = 0 all joints are aligned and the position can be checked by hand. The result of the code is

$$\begin{vmatrix} 1 & 0 & 0 & -0.8172 \\ 0 & 0 & -1 & -0.2329 \\ 0 & 1 & 0 & 0.0628 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

T_{06} at q = 0

This agrees with the geometry: $x = a_2 + a_3 = -0.4250 - 0.3922 = -0.8172$ m; $y = -(d_4 + d_6) = -(0.1333 + 0.0996) = -0.2329$ m; and $z = d_1 - d_5 = 0.1625 - 0.0997 = 0.0628$ m. Matching a hand calculation on the zero pose confirms that the DH table and the matrix formula were entered correctly.

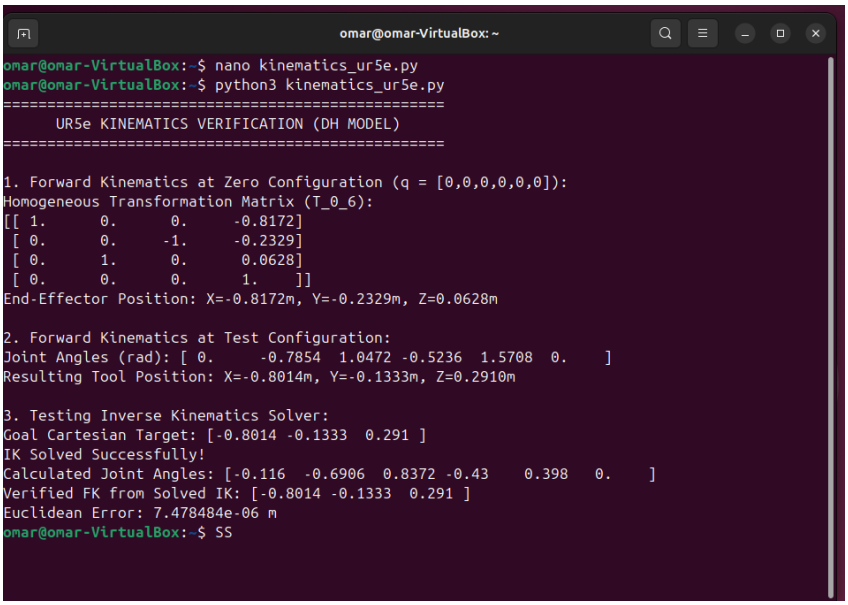


Figure 3. Output of kinematics_ur5e.py showing the zero-configuration transform, the test-configuration FK and the IK verification.

5.2 Verification table

Configuration	Joint angles [$\theta_1 \ldots \theta_6$] (rad)	End-effector (X, Y, Z) (m)
Zero reference	[0, 0, 0, 0, 0, 0]	(−0.8172, −0.2329, 0.0628)
Arbitrary test	[0, −0.7854, 1.0472, −0.5236, 1.5708, 0]	(−0.8014, −0.1333, 0.2910)
Upright	[0, −1.5708, 0, −1.5708, 0, 0]	(0.0000, −0.2329, 1.0794)
Reach	[0, −1.0, 1.2, −0.5, 1.57, 0]	(−0.7386, −0.1334, 0.3764)
Base turn	[1.57, −1.0, 1.2, −0.5, 1.57, 0]	(0.1328, −0.7387, 0.3764)

Table 2. Forward kinematics results. The last three rows are the presets used by the GUI and the test script. The upright pose has x = 0 because the arm is vertical; "base turn" differs from "reach" only by a 90° rotation about the base axis, which swaps the horizontal coordinates while leaving Z unchanged (0.3764 m in both), as expected.

6. Inverse position kinematics

6.1 Method

A closed-form solution is possible for the UR5e, but this milestone requires position kinematics only, so a numerical solver was implemented. The solver uses damped least squares (Levenberg–Marquardt). At each iteration the position error e = p* − p(q) is computed from the forward kinematics, the 3×6 position Jacobian J is estimated numerically with central differences, and the joint update is

$$\Delta q = J^{T} \left(J J^{T} + \lambda^{2} I\right)^{-1} e\,, \quad q \leftarrow q + \Delta q$$

The damping factor λ = 0.01 keeps the update bounded near singular configurations. The loop stops when $\|e\| < 10^{-4}$ m or after 200 iterations, and the final angles are wrapped into [−π, π]. The function returns the joint vector and a success flag.

Parameter	Value in kinematics_ur5e.py
Tolerance	1e-4 m
Maximum iterations	200
Damping λ	0.01

Parameter	Value in kinematics_ur5e.py
Default initial guess	[0, −π/2, π/2, 0, 0, 0]
Jacobian	Numerical, central difference, ε = 1e-6

6.2 Result for the test target

- Target position: [−0.8014, −0.1333, 0.2910] m
- Computed joint angles: [−0.1160, −0.6906, 0.8372, −0.4300, 0.3980, 0.0] rad
- Reconstructed FK position: [−0.8014, −0.1333, 0.2910] m
- Position error: 7.478×10^{-6} m

The solved angles differ from the angles used to create the target. This is expected: a position target fixes only three of the six degrees of freedom, so infinitely many joint configurations reach the same point. The solver converges to the one closest to its initial guess, and θ_6 is unchanged because rotating the last joint does not move the flange origin.

6.3 Robustness test

To check that the solver is not tuned to a single target, 200 random joint vectors were drawn uniformly from $[-\pi, \pi]^6$, their positions were computed with the forward kinematics, and the solver was started from the default initial guess. All 200 targets converged (100 %), with a median residual of 5.3×10^{-6} m and a maximum of 9.9×10^{-5} m, below the 10^{-4} m tolerance. This test only covers reachability of positions generated by the same model; it does not test joint limits or collisions.

6.4 Limitations

- Position only: the tool orientation is not controlled.
- The result depends on the initial guess; the solver returns a local solution and does not enumerate the up to eight analytical UR5e solutions.
- Joint limits and self-collision are not checked. The GUI version limits the step size to 0.2 rad per iteration and uses 40 iterations with a 1 mm tolerance for responsiveness.
- Targets outside the reachable workspace (distance from the shoulder greater than about 0.85 m) will not converge.

7. Simulator linking: ROS 2 and MuJoCo

The UR5e model is loaded in MuJoCo and connected to ROS 2 Jazzy through a single Python node, `UR5eActuationController` (`mujoco_actuation_ur5e.py`), built with `rclpy` and the `mujoco` bindings. The node and the physics run in the same process, which avoids any extra communication layer between ROS and the simulator.

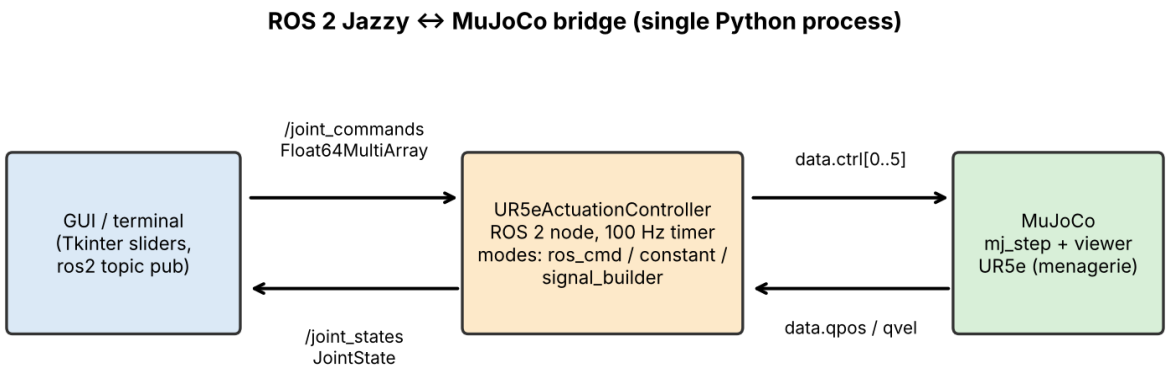


Figure 4. Architecture of the bridge and the topics it uses.

Topic	Message type	Use
/joint_commands	std_msgs/Float64MultiArray (6 values)	Subscribed. Joint targets in radians, order: pan, lift, elbow, wrist 1, 2, 3.
/joint_states	sensor_msgs/JointState	Published every 10 ms. Joint names, positions (qpos) and velocities (qvel) from MuJoCo.

7.1 Data flow

1. **Physics stepping:** the main loop calls `mujoco.mj_step`, then `rclpy.spin_once` (non-blocking), then `viewer.sync()`, and sleeps for the remainder of the model time step so the simulation runs close to real time.
2. **Command handling:** the subscriber callback stores a valid 6-element message as the new target (`target_qpos`).
3. **Control loop:** a ROS timer at 100 Hz (0.01 s) selects the command according to the active mode and writes it to `data.ctrl[0..5]` (the position actuators). If the model has fewer than six actuators, the code writes `qpos` directly as a kinematic fallback.
4. **Telemetry:** the same timer publishes a `JointState` message with the current positions and velocities.
5. **Verification:** `ros2 topic echo /joint_states --once` was used to confirm that all six joints are streamed (Figure 5), and `ros2 topic pub --once /joint_commands` was used to send a pose (Figure 6).

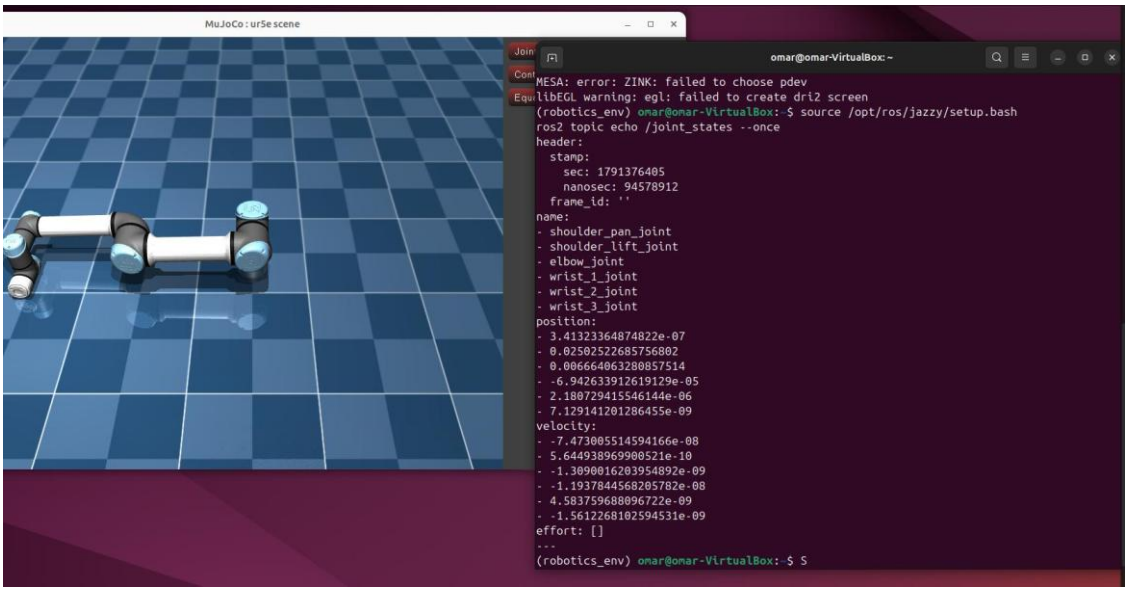


Figure 5. MuJoCo viewer with the bridge running and ros2 topic echo /joint_states --once showing the six joint names, positions and (near-zero) velocities at rest.

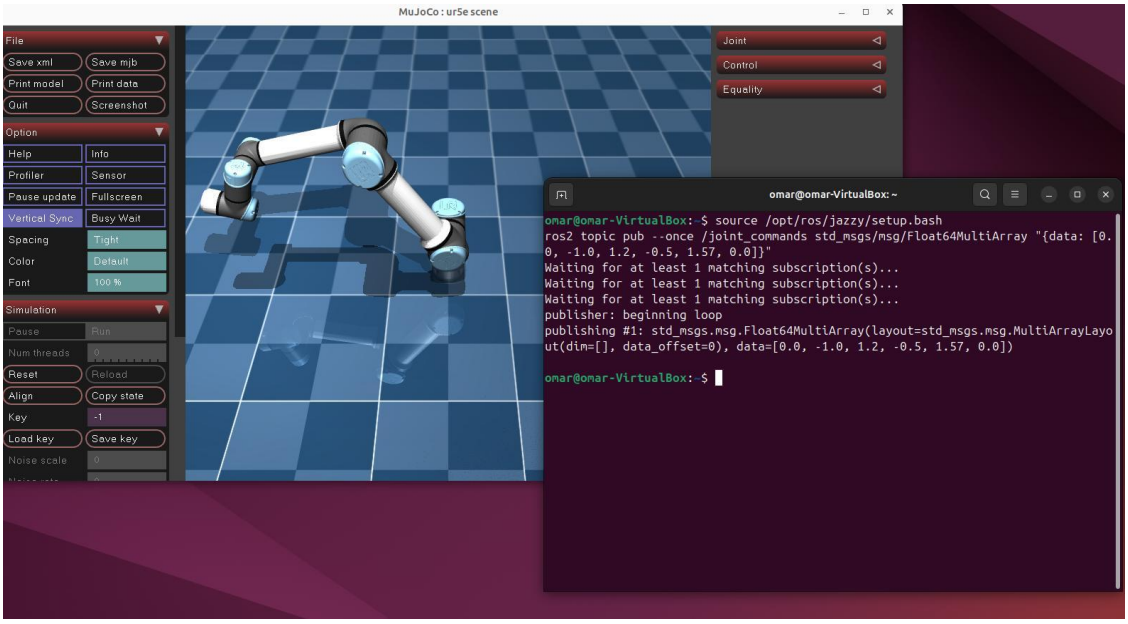


Figure 6. A command published from a terminal to /joint_commands: [0.0, −1.0, 1.2, −0.5, 1.57, 0.0]. The arm moves to the reach pose in the viewer.

Note on rates: the timer period in the code is 0.01 s, so telemetry is published at 100 Hz.

8. Motion inputs and analysis

The bridge has three input modes, selected by the variable `self.mode`, plus slider inputs from the GUI and from the MuJoCo viewer.

Input	Description
Constant	A fixed posture [0, −π/3, π/4, −π/6, π/2, 0] is written to the actuators every cycle (a step input from the initial pose).
ros_cmd (default)	Joint targets arrive from any ROS 2 publisher: the terminal, the GUI sliders or the preset buttons.
signal_builder	Smooth harmonic joint commands generated inside the node (Table 3).

Input	Description
Sliders	Tkinter joint sliders (and Cartesian sliders in the IK GUI) publish to /joint_commands; the MuJoCo viewer's own control panel also drives the actuators.

Joint	Command q(t)	Amplitude	Frequency
Shoulder pan	0.6 sin(0.5 t)	0.6 rad	0.5 rad/s
Shoulder lift	−π/4 + 0.3 sin(0.4 t)	0.3 rad	0.4 rad/s
Elbow	π/3 + 0.4 cos(0.4 t)	0.4 rad	0.4 rad/s
Wrist 1	−π/4 + 0.2 sin(0.6 t)	0.2 rad	0.6 rad/s
Wrist 2	0.3 sin(0.8 t)	0.3 rad	0.8 rad/s
Wrist 3	0.5 sin(1.0 t)	0.5 rad	1.0 rad/s

Table 3. Signal-builder commands. Different frequencies make the combined motion non-repeating over short times.

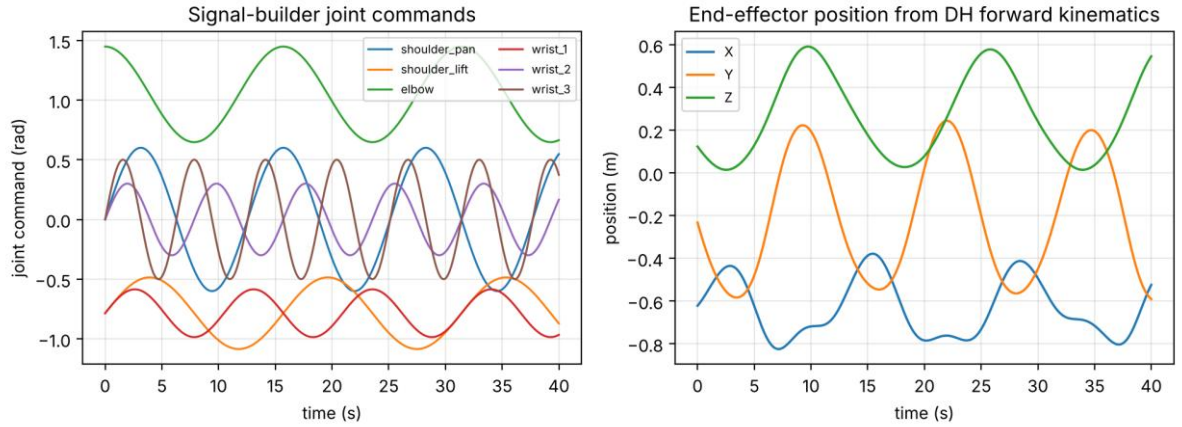


Figure 7. Left: the six signal-builder joint commands over 40 s. Right: the corresponding end-effector position computed with the DH forward kinematics.

Analysis: over 40 s the commanded end effector (in the DH base frame) moves within X ∈ [−0.825, −0.379] m, Y ∈ [−0.591, 0.244] m and Z ∈ [0.014, 0.591] m. The minimum height of 0.014 m means the tool flange comes very close to the base plane (z = 0). That is acceptable for a bare arm in free space, but once a table, conveyor or other environment objects are added, the amplitudes or offsets of the lift and elbow signals will need to be reduced to avoid contact. The commands are smooth, so the position actuators receive no steps after the first sample, and velocity changes are limited.

9. GUI development

Two Tkinter control panels were written. Both are ROS 2 nodes that publish joint targets to /joint_commands and subscribe to /joint_states, and both call `rclpy.spin_once` from Tkinter's `after()` timer every 20 ms so that the GUI stays responsive.

Feature	Basic panel	IK panel
File	`ur5e_gui.py`	`New_ur5e_gui.py`
Window title	UR5e ROS 2 Control Panel	UR5e IK Cartesian & Joint Controller

Feature	Basic panel	IK panel
Joint sliders	6 sliders, −3.1416 to 3.1416 rad	6 sliders, same range, synchronised with Cartesian values
Cartesian sliders	None	X, Y, Z sliders; each move solves IK and updates the joints
Presets	Zero Pose, Upright, Reach	Reset Upright, Home / Ready
Kinematics used	None (joint-space only)	DH forward kinematics and damped least squares IK

In the IK panel, moving a joint slider recomputes the end-effector position with the forward kinematics and updates the Cartesian sliders; moving a Cartesian slider runs the inverse kinematics from the current joint angles and updates the joint sliders. A flag (`updating_from_code`) prevents the two directions from triggering each other in a loop.

9.1 Known limitations and planned work

- The Cartesian coordinates shown in the GUI are expressed in the DH base frame. The MuJoCo scene may place the robot base differently, so the GUI position and the position of the tool in the viewer may differ by a fixed offset. Comparing displacements from the zero pose, which do not depend on the base frame, is a way to measure this offset.
- Sliders are limited to ±π whereas the UR5e joints allow about ±2π; limits should be read from the model.
- The IK panel only controls position; an orientation input is needed for tool tasks.
- Milestone 03 will add the environment components (conveyor, objects, table) around the arm and the velocity-level GUI features.

10. Conclusions

The UR5e was modelled with standard DH parameters and its forward kinematics reproduce a hand calculation at the zero pose. The numerical inverse kinematics reached all 200 random reachable positions to within 10^{-4} m, and the model runs in MuJoCo linked to ROS 2 Jazzy through topics that were verified from the terminal. The robot can be driven by constants, signal-builder trajectories and sliders, and the first two GUI panels are working. The main open items are quantifying the offset between the DH frame and the MuJoCo frame, adding orientation to the inverse kinematics, and building the industrial environment for the next milestones.

Appendix A. Files delivered in the Milestone 02 folder

File	Purpose
`kinematics_ur5e.py`	DH table, FK, numerical Jacobian, damped least squares IK, verification printout
`mujoco_actuation_ur5e.py`	ROS 2 – MuJoCo bridge node with constant, signal-builder and ROS command modes
`ur5e_gui.py`	Joint-space GUI with presets
`New_ur5e_gui.py`	Cartesian IK and joint GUI

File	Purpose
Video	Simulation, motion inputs and GUI building process (uploaded in the same GitHub folder)
This report	DH convention, forward kinematics and analysis